

MAEP: A Minimum Viable Agent-to-Agent Economic Protocol with Decentralized Identity and Trustless Payment Settlement

Anonymous Author(s)
Submitted for Review
2026

Abstract—Existing agent communication protocols such as Google A2A and Coinbase x402 lack two critical primitives required for fully autonomous economic activity: verifiable decentralized agent identity and trustless micropayment settlement with on-chain result verification. We present MAEP (Minimum Viable Agent-to-Agent Economic Protocol), a five-stage protocol that addresses these gaps through three Ethereum smart contracts—`AgentRegistry`, `PaymentChannel`, and `ResultAttestation`—and a Python Agent SDK with plug-gable LLM backends. We evaluate MAEP against x402, Google A2A, and Fetch.ai on three dimensions: end-to-end latency overhead, on-chain Gas cost, and trust assumption count. Our experiments show that (1) MAEP reduces trust assumptions to a single party (Ethereum consensus) versus 2–3 for competing protocols; (2) the total Gas cost for one complete task delegation is 408,870 units on Base Sepolia, equating to a negligible transaction fee at Layer-2 prices; and (3) the protocol state-machine itself adds only microsecond-level latency above plain function calls. An end-to-end three-agent demo—including a contested dispute resolved by an `AuditorAgent`—completes in under five seconds. We release the full implementation as open-source software.

Index Terms—agent-to-agent protocol, decentralized identity, smart contracts, micropayment, trustless settlement, large language models, Web3

I. INTRODUCTION

The emergence of large language model (LLM)-powered autonomous agents has shifted the question of AI coordination from theoretical interest to practical urgency. Agents are no longer passive tools operated by humans; they are increasingly capable of planning, tool use, and sustained goal pursuit [1]. A natural next step is for agents to *hire* one another: a Requester agent decomposes a task, discovers a specialized Provider agent, delegates work, and pays for the result—all without human intervention.

This vision requires two primitives that current protocols do not provide together:

- 1) **Verifiable agent identity.** How does a requester know that the provider it found is who it claims to be, and that its declared capabilities are genuine? Centralized registries (Google A2A, platform-hosted agent cards) answer this by trusting the platform operator. For truly autonomous, cross-organizational interaction, this is insufficient.
- 2) **Trustless payment settlement.** How can a requester pay for a result without either (a) paying upfront and risking non-delivery, or (b) waiting for a human to authorize payment? Existing HTTP-based micropayment schemes (e.g., x402) rely on stablecoin infrastructure and server-side honesty.

We propose MAEP, a Minimum Viable Agent-to-Agent Economic Protocol that resolves both problems using public smart contracts on an EVM-compatible blockchain. MAEP is deliberately minimal: it does the least possible on-chain work necessary to remove the need for a trusted intermediary, while delegating expensive computation (LLM inference, task matching) to off-chain agents.

The main contributions of this paper are:

- A formal five-stage protocol specification (Register, Discover, Delegate, Execute, Settle) with defined state transitions, message formats, and security invariants (§IV).
- A reference implementation comprising three audited Solidity contracts and a Python SDK supporting six LLM providers (§V).
- A comparative evaluation on latency, Gas cost, and trust assumption count, demonstrating that MAEP achieves the lowest trust footprint among surveyed protocols (§VI).

II. BACKGROUND

This section introduces the foundational concepts required to understand MAEP. Readers familiar with blockchain and A2A protocols may skim to §III.

A. Blockchain and Smart Contracts

A *blockchain* is a distributed ledger replicated across thousands of nodes. Transactions are grouped into blocks, and each block cryptographically references its predecessor, forming a tamper-evident chain. The key property relevant here is *trustless execution*: any computation defined in a smart contract executes identically on every node, and the result is final once confirmed.

Smart contracts are programs stored on-chain. They have a persistent address, can hold cryptocurrency balances, and

expose functions callable by any external account. Crucially, their source code is public and their execution is deterministic—no operator can alter behavior after deployment.

The *Ethereum Virtual Machine* (EVM) is the standard runtime environment for smart contracts. An instruction unit called *Gas* prices each EVM operation; users pay Gas fees in the network’s native currency (ETH) to execute transactions. Gas prices fluctuate with network demand, but Gas *units* consumed by a given computation are deterministic—the same code path always costs the same number of Gas units regardless of when or where it is executed.

Layer 2 (L2) networks (such as Base, Arbitrum, and Optimism) run the same EVM but batch transactions and post proofs to Ethereum mainnet, reducing fees by two to three orders of magnitude. MAEP targets **Base**, a Coinbase-operated Optimistic Rollup, because its tooling is mature and its gas prices are typically below 0.01 gwei—making sub-cent transaction fees realistic.

B. Cryptographic Hash Functions

A *hash function* maps arbitrary-length data to a fixed-length digest. The `keccak256` function used in Ethereum is a one-way function: it is computationally infeasible to recover the input from the digest, and any change to the input changes the digest completely. MAEP uses `keccak256(result_data)` as a *commitment*: the provider posts only the hash on-chain (cheap), and later reveals the full result off-chain. The requester verifies that `keccak256(received_data) == on_chain_hash`.

C. Escrow and Payment Channels

An *escrow* is a neutral holding mechanism: funds are locked by a third party and released only when predefined conditions are met. Traditional escrow requires a trusted third party (a bank, a lawyer). Smart-contract escrow replaces the trusted party with code: funds are locked in the contract’s balance and released when the verification logic returns true.

MAEP’s `PaymentChannel` contract implements a simple escrow: (1) the requester calls `lock(taskId, provider, deadline)` with attached ETH; (2) if the provider submits a result and the requester confirms, `release(taskId)` transfers funds to the provider; (3) if the deadline passes without settlement, `refund(taskId)` returns funds to the requester. No human intervention is required at any step.

D. Decentralized Identity (DID)

Decentralized Identifiers (DIDs) are globally unique identifiers controlled by their subject rather than by a registrar. The W3C DID specification [2] defines a data model; different DID methods specify how identifiers are created and resolved on different substrates (IPFS, Bitcoin, Ethereum, etc.).

MAEP implements a lightweight on-chain DID registry. Each agent registers a JSON-encoded capability document

(e.g., `{"task_types":["data_analysis"]}`) alongside a minimum stake in ETH. The stake creates a mild Sybil-resistance mechanism: an attacker registering N fake identities must lock $N \times \text{MIN_STAKE}$ ETH. Reputation scores are updated on-chain after each settlement, providing a persistent, tamper-evident track record.

E. Agent-to-Agent (A2A) Communication

An *A2A protocol* defines how autonomous agents discover each other, negotiate tasks, and exchange results. The essential components are:

- **Discovery**: how agents find peers with needed capabilities.
- **Task specification**: a shared format for describing what is needed and at what price.
- **Result delivery**: how the provider returns its output.
- **Settlement**: how payment flows after task completion.

Current proposals (Google A2A, x402, Fetch.ai) provide subsets of these components. MAEP aims to be the minimal protocol that covers all four with trust-minimized guarantees.

F. Large Language Model Agents

An *LLM agent* wraps a language model with a control loop that allows it to use tools, maintain state, and pursue multi-step goals. In MAEP’s reference implementation, agents are Python objects that hold an `LLMClient` (which abstracts over different providers) and a `MAEPSession` (which tracks protocol state). When a provider agent receives a task, it constructs a prompt, calls its LLM backend, and returns the generated text as the result. The SDK is deliberately LLM-agnostic: the same agent code works with OpenAI GPT, Anthropic Claude, local Ollama models, or any OpenAI-compatible endpoint.

III. RELATED WORK

A. Google Agent2Agent (A2A) Protocol

1) *Overview and Motivation*: Google released the Agent2Agent (A2A) protocol in April 2025 as an open standard for interoperability between AI agents built on different platforms [3]. The driving observation is that enterprise AI deployments increasingly involve agents from multiple vendors (e.g., a Salesforce CRM agent delegating to a SAP logistics agent), and without a shared communication standard each pair of vendors must negotiate a custom integration. A2A defines a common JSON-over-HTTP wire format so that any compliant agent can talk to any other without pre-arranged bilateral agreements at the *communication level*.

2) *Core Mechanism: Agent Cards and Task Lifecycle*: The central abstraction in A2A is the **Agent Card**—a JSON document served at a well-known HTTPS endpoint (e.g., `https://agent.example.com/.well-known/agent.json`). An Agent Card declares:

- **Identity fields**: agent name, description, version, and the URL of the agent’s A2A endpoint.

- **Capability list:** a set of *skills*, each with a natural-language description, input/output schema, and optional example messages.
- **Authentication requirements:** which OAuth 2.0 scopes or API keys the caller must present.
- **Supported modalities:** whether the agent can handle text, file attachments, structured data, audio, or video.

A **Requester** agent (called the *client* in A2A terminology) discovers a Provider agent (the *remote agent*) by fetching its Agent Card, inspects the declared skills, and initiates a **Task**. The task lifecycle follows four states:

- 1) *submitted* – the client sends a `tasks/send` JSON-RPC call containing the task input and a unique task ID.
- 2) *working* – the remote agent is processing; it may stream intermediate *artifacts* (partial results) back to the client via Server-Sent Events (SSE) or WebSockets.
- 3) *input-required* – the remote agent needs clarification and pauses to request additional user or agent input (supporting multi-turn interaction).
- 4) *completed / failed / canceled* – terminal states returned with final artifacts or error codes.

All messages are JSON objects transported over HTTPS, making A2A easy to implement in any language and compatible with existing API infrastructure.

3) **Identity and Trust Model: How A2A establishes identity.** In A2A, an agent’s identity is anchored to its *HTTPS domain*. When Agent B serves its Agent Card at `https://b.example.com/.well-known/agent.json`, Agent A trusts that the card genuinely represents the operator of `b.example.com` because HTTPS certificate validation guarantees the domain owner controls the TLS private key. This is the same trust model as ordinary Web PKI: the browser trusts a certificate because a Certificate Authority (CA) has vouched for the domain.

Limitations of domain-anchored identity. This approach has three concrete weaknesses relevant to autonomous agent economies:

- 1) **Platform operator trust.** For agents hosted on a managed platform (e.g., Google Agentspace, Microsoft Copilot Studio), the Agent Card is served by the *platform*, not by the agent itself. If the platform operator modifies the declared capabilities, censors an agent, or revokes access, the requester has no cryptographic means to detect the change. The requester trusts the platform operator by necessity.
- 2) **Capability claims are unverified.** An Agent Card declares skills in natural language. There is no mechanism in the A2A protocol to *attest* that a provider has successfully completed tasks of the declared type in the past, or to penalize it for false claims. A malicious provider can register arbitrary capabilities with no cost and no accountability.
- 3) **No cross-organizational reputation.** A2A provides no persistent reputation or history mechanism. If an agent

from Company X fails to deliver on a task for Company Y, Company Y has no standardized way to record this failure in a location visible to future requesters from Company Z. Each organization must maintain its own private allow/deny list.

Authentication (not the same as identity). A2A supports OAuth 2.0, API keys, and other standard HTTP authentication schemes. This authenticates *who is calling* at the transport level, but does not address the deeper question of whether the callee’s declared capabilities are trustworthy. Authentication proves “this request comes from account X”; it does not prove “account Y (the callee) has the skills it claims and will honor its commitments.”

4) **Economic Primitives: Deliberately Absent:** A2A is explicitly scoped to *communication*, not economics. The protocol specification states that payment is out of scope and should be handled by “existing payment infrastructure.” Concretely:

- There is **no escrow**: a requester who pre-pays has no protocol-level recourse if the provider returns garbage.
- There is **no result verification**: the protocol delivers artifacts but does not commit to their hash or provide any proof of correct computation.
- There is **no dispute resolution**: the specification does not define what happens when the requester rejects a result.

This is a reasonable scope decision for A2A’s primary use case (enterprise integration within trusted organizational boundaries), but it means A2A is insufficient as a substrate for open, permissionless agent markets where requesters and providers may have no prior relationship and no shared institutional accountability.

5) **Comparison with MAEP:** MAEP treats A2A as a *complementary* protocol rather than a competitor. A2A’s message format and task lifecycle could serve as the off-chain communication layer for MAEP’s DISCOVER and EXECUTE stages—agents would find each other and exchange task details using A2A messages, while using MAEP’s smart contracts for binding identity registration, payment escrow, and result attestation. In this view, MAEP adds the *economic trust layer* that A2A deliberately omits.

B. Coinbase x402

x402 [4] extends HTTP with a 402 Payment Required response and a standardized payment header. The client prepares an on-chain USDC transfer; the server verifies the transaction and proceeds. x402 solves the *pay-per-request* problem elegantly for server-side APIs, but it does not provide agent identity, result verification, or dispute resolution. Payment flows from client to server without any escrow; if the server misbehaves, the client has no recourse within the protocol.

C. Fetch.ai / Agentverse

Fetch.ai [5] provides a full ecosystem for autonomous economic agents (AEAs) built on its own Cosmos-based blockchain. Agents use the FET token for payment, and the network handles identity and discovery. Fetch.ai is more

feature-complete than MAEP for in-ecosystem agents, but it requires trust in the Fetch.ai validator set and governance, which is a permissioned subset of the broader Ethereum trust base. MAEP runs on Ethereum/Base, which has a larger, more decentralized validator set.

D. Autonolas / Olas

Autonolas [6] defines on-chain service registries and multi-agent service coordination, with staking and slashing for misbehaving agents. Olas is a production system with real staking economics. MAEP is complementary: where Olas focuses on complex service orchestration with DAOs, MAEP targets the simpler bilateral task delegation primitive in a minimally invasive way.

E. Recent Academic Work

Closest to our work is a concurrent preprint by Andres et al. [7], which proposes extending A2A with blockchain identity and payments. Their design is conceptually aligned with ours; MAEP differs in providing (a) a complete formal protocol specification, (b) a working open-source implementation with quantitative experiments, and (c) an explicit trust-minimization analysis.

IV. MAEP PROTOCOL SPECIFICATION

A. Threat Model and Design Goals

We consider two rational, potentially self-interested parties: **Requester** and **Provider**. Neither is assumed to be honest, but both are assumed to be economically rational (they will not take actions that cost them more than they gain).

Threat model:

- Provider may claim false capabilities at registration.
- Provider may return garbage results after receiving payment.
- Requester may refuse to acknowledge a valid result to avoid payment.
- Either party may abandon the protocol mid-execution.

Design goals:

- 1) **G1 – Payment safety:** the requester cannot lose funds without receiving a committed result hash.
- 2) **G2 – Liveness:** if both parties act honestly, the protocol always terminates in SETTLED.
- 3) **G3 – Sybil resistance:** creating fake identities has a non-zero cost.
- 4) **G4 – Minimal trust:** the only trusted party is the EVM’s execution guarantee.

B. Five-Stage Protocol Flow

Table I summarizes the five stages. Figure ?? illustrates the message sequence.

Stage 1 – REGISTER. An agent calls `AgentRegistry.register(capabilityJSON)` with a minimum stake of 0.0001 ETH attached. The contract stores the agent’s address, capability document, reputation score (initialized to 100), and stake. The stake is returned on deregistration if reputation remains above a threshold.

TABLE I
MAEP PROTOCOL STAGES

Stage	Actor	On-chain?	Key Action
1. REGISTER	Both	Yes	Submit DID + capabilities + stake
2. DISCOVER	Requester	No	Match by task type + budget
3. DELEGATE	Requester	Yes	Lock ETH in escrow; send task spec
4. EXECUTE	Provider	Yes	Submit <code>keccak256(result)</code>
5. SETTLE	Both	Yes	Verify; release payment; update rep.

Stage 2 – DISCOVER. The requester queries the off-chain Python SDK’s `discover()` method, which reads the on-chain registry and filters agents whose declared `task_types` include the required type and whose reputation exceeds a configurable minimum. Discovery is off-chain to avoid Gas overhead for what may be a frequently repeated read operation. The on-chain registry remains the authoritative source of truth.

Stage 3 – DELEGATE. The requester calls `PaymentChannel.lock(taskId, providerAddress, deadline)` with the agreed payment attached as ETH. The contract records:

`locks[taskId] = (requester, provider, amount, deadline, LOCKED)`

The requester then sends the full task specification to the provider over any off-chain channel (HTTP, WebSocket, etc.). The `taskId` serves as the binding link between the off-chain task and the on-chain escrow.

Stage 4 – EXECUTE. The provider processes the task using its LLM backend and calls `ResultAttestation.attest(taskId, keccak256(resultData))`. This commits the result on-chain before revealing the full data, preventing the provider from changing the result after seeing the requester’s reaction (*commit-reveal pattern*). The provider then delivers `resultData` to the requester off-chain.

Stage 5 – SETTLE. The requester verifies that `keccak256(received)` matches the on-chain attestation. If accepted, it calls `PaymentChannel.release(taskId)`, which transfers ETH to the provider and increments the provider’s reputation. If rejected, the session enters DISPUTED state and escalates to an `AuditorAgent`, which independently evaluates both the task specification and the result and issues a ruling. If no settlement occurs before the deadline, either party can call `PaymentChannel.refund(taskId)` to recover the locked funds.

C. Security Analysis

G1 (Payment safety). Funds are transferred only in `release()` or `refund()`. The contract follows the Checks-Effects-Interactions (CEI) pattern [8]: state is updated before any external call, preventing reentrancy attacks.

G2 (Liveness). If both parties act honestly, the provider submits a valid result, the requester’s verification passes, and `release()` is called. Two failure modes are covered by the protocol:

Unresponsive provider. If the provider never submits a result, the deadline mechanism protects the requester: after the deadline expires, the requester calls `refund(taskId)` to recover the locked ETH. No human intervention is required; the contract enforces the timeout autonomously.

Unresponsive requester. If the provider submits a valid result hash but the requester neither calls `release()` nor `dispute()`, the funds remain locked until the deadline passes, at which point the requester may reclaim them via `refund()`. This is a known asymmetry in MAEP’s current design: a requester can effectively withhold payment without triggering a dispute, so long as it accepts the deadline penalty. Mitigations include requiring the requester to explicitly reject (triggering mandatory arbitration) or implementing an auto-release after a response deadline—both are left as future work (see §VII).

G3 (Sybil resistance). Each registration requires a minimum stake. An adversary creating N identities locks $N \times 0.0001$ ETH, providing economic friction without prohibitive cost for legitimate agents.

G4 (Minimal trust). Every conditional fund transfer is governed by contract logic, not by any operator. The `AuditorAgent` is the only subjective component; its ruling could be replaced by a DAO vote or a cryptographic oracle without changing the protocol’s on-chain structure.

V. IMPLEMENTATION

A. Smart Contracts

The three contracts are implemented in Solidity 0.8.24 and compiled with the Hardhat 3 development framework. The full source is available in the open-source repository.

AgentRegistry.sol (key interface):

```
function register(string calldata capabilities)
    external payable;

function updateReputation(address agent, int8 delta)
    external; // callable by PaymentChannel only

function deregister() external;
```

Listing 1. AgentRegistry interface

Reputation is stored as a `uint16` bounded to $[0, 10000]$. The `updateReputation` function is access-controlled to the `PaymentChannel` address set at deployment, preventing arbitrary manipulation.

PaymentChannel.sol (key interface):

```
function lock(bytes32 taskId,
    address provider,
    uint256 deadline)
    external payable;

function release(bytes32 taskId) external;

function refund(bytes32 taskId) external;
```

Listing 2. PaymentChannel interface

ETH transfers use the low-level `.call{value}("")` pattern rather than `.transfer()` to avoid gas stipend issues with smart contract recipients.

ResultAttestation.sol (key interface):

```
function attest(bytes32 taskId, bytes32 resultHash)
    external;

function dispute(bytes32 taskId) external;

function getAttestation(bytes32 taskId)
    external view
    returns (bytes32 hash, address attester,
        uint256 timestamp, bool disputed);
```

Listing 3. ResultAttestation interface

B. Python Agent SDK

The SDK is structured around four modules:

- `config.py`: reads `LLM_PROVIDER`, `LLM_API_KEY`, `LLM_MODEL`, and `LLM_BASE_URL` from a `.env` file.
- `llm_client.py`: a unified wrapper supporting OpenAI, Anthropic, Zhipu, MiniMax, Ollama, and any OpenAI-compatible endpoint. The `complete(prompt)` method transparently handles provider-specific authentication and response parsing, including fallback from empty content to reasoning fields for chain-of-thought models.
- `protocol.py`: the `MAEPSession` state machine and `TaskSpec / TaskResult` dataclasses.
- `requester.py`, `provider.py`, `auditor.py`: agent roles that compose an `LLMClient` with protocol operations.

C. Deployment

Contracts are deployed using a Hardhat deployment script with `ethers.js`. Deployed addresses on Base Sepolia (chain ID 84532) are:

- AgentRegistry: `0xc4146C22...d7445`
- PaymentChannel: `0x4fA1a014...7794`
- ResultAttestation: `0xC915A7e9...f461`

VI. EVALUATION

We evaluate MAEP along four axes, corresponding to the four experiments described below. All experiments were run on a Windows 11 machine (Intel i7, 16 GB RAM) with Hardhat 3 for local measurements and the public Base Sepolia RPC endpoint for on-chain measurements.

A. Experiment 1: Latency Overhead

Goal. Quantify the latency of MAEP’s protocol operations at two levels: (1) off-chain state-machine overhead versus equivalent bare Python operations; and (2) on-chain confirmation latency per stage on a local Hardhat network.

Setup. We use a two-table design to separate the two fundamentally different latency sources.

Table II – Off-chain overhead. We ran $n = 200$ iterations of each stage in two configurations. **MAEP:** each stage is timed in isolation on a pre-built session (prior stages completed but not re-timed), so measurements are not cumulative. **Baseline:** the equivalent bare Python operations that a plain HTTP A2A client would perform at the same stage—specifically,

dict construction for DELEGATE, a `hashlib.sha256` call plus dict assignment for EXECUTE (matching MAEP’s hash commitment step), and a boolean branch plus dict update for SETTLE. The baseline thus reflects the irreducible work any protocol must do at each stage, making the comparison apples-to-apples rather than MAEP-vs.-nothing. All measurements use `time.perf_counter_ns()` and are local in-process to isolate from network and blockchain latency.

Table III – On-chain confirmation latency. We ran $n = 5$ full MAEP task lifecycles against a local Hardhat node, timing each stage from `send_raw_transaction()` to `wait_for_transaction_receipt()`. This captures real EVM execution plus JSON-RPC round-trip overhead, and represents the minimum achievable on-chain latency (no real network propagation).

Results. Tables II and III report the results.

TABLE II

OFF-CHAIN PROTOCOL OVERHEAD PER STAGE ($n = 200, \mu\text{s}$). EACH STAGE IS TIMED IN ISOLATION ON A PRE-BUILT SESSION. BASELINE = EQUIVALENT BARE PYTHON OPERATIONS (DICT, SHA256, BRANCH).

Stage	MAEP mean	Base mean	Overhead	MAEP σ
DELEGATE	1.12	0.23	4.8×	0.47
EXECUTE	3.32	1.08	3.1×	2.91
SETTLE	1.47	0.22	6.8×	0.27

TABLE III

ON-CHAIN CONFIRMATION LATENCY ON HARDHAT LOCAL NETWORK ($n = 5$, MS, MEAN $\pm$ STD).

Stage	Mean (ms)	Std (ms)	Range (ms)
REGISTER	11.2	9.5	5.1–28.0
DELEGATE	7.8	2.5	6.0–12.2
EXECUTE	5.5	0.5	4.6–6.1
SETTLE	6.0	0.4	5.5–6.4
TOTAL	30.5	–	–

Analysis. *Off-chain overhead (Table II).* The MAEP state machine adds 1–3.3 μs per stage above the equivalent bare operations. The EXECUTE stage is the most expensive because it both computes a hash and performs a dataclass instantiation; its high standard deviation ($\sigma = 2.91 \mu\text{s}$) reflects CPU cache effects across 200 runs. The SETTLE stage shows the highest relative overhead (6.8×) because its baseline—a boolean branch—is extremely cheap; in absolute terms the overhead is only 1.25 μs . All values are sub-microsecond to low-microsecond, confirming that the protocol state machine is not a performance bottleneck.

On-chain confirmation latency (Table III). On Hardhat local, the four on-chain stages complete in 30.5 ms total on average. The REGISTER stage has the highest variance (std 9.5 ms) because it involves a larger storage write (JSON capability string), causing occasional JIT compilation effects in the EVM interpreter. On a real network such as Base Sepolia, each stage requires waiting for a block (typically 2 seconds), making the total per-task on-chain latency approximately 8 seconds—four

to five orders of magnitude larger than the off-chain state-machine overhead measured in Table II. This confirms that MAEP’s design decision to perform DISCOVER off-chain is well-motivated: moving even one additional read operation on-chain would add seconds of latency with no security benefit.

B. Experiment 2: Gas Cost Analysis

Goal. Measure on-chain Gas consumption per stage to assess economic viability and guide future optimization.

Setup. We ran the full MAEP five-stage flow on both a local Hardhat network (deterministic, zero-latency) and the Base Sepolia testnet (chain ID 84532). Because EVM Gas units are fully deterministic for a given bytecode and call data, local and testnet values are expected to match. We report both for validation.

Results. Table IV reports Gas units per stage.

TABLE IV

GAS CONSUMPTION PER STAGE: LOCAL HARDHAT VS. BASE SEPOLIA

Stage	Hardhat	Base Sepolia	Match
REGISTER	138,625	138,625	Exact
DELEGATE	116,489	116,501	$\Delta+12$
EXECUTE	93,590	93,602	$\Delta+12$
SETTLE	62,642	60,142	$\Delta-2,500$
TOTAL	411,346	408,870	$\approx 0.6\%$ diff

Table V reports one-time contract deployment costs.

TABLE V

CONTRACT DEPLOYMENT GAS (ONE-TIME)

Contract	Gas Units
AgentRegistry	1,234,274
PaymentChannel	1,161,550
ResultAttestation	516,032
Total deployment	2,911,856

Analysis. Gas consumption decreases monotonically from REGISTER to SETTLE, reflecting the storage-write-heavy nature of early stages and the storage-read-plus-transfer nature of the final stage. The small discrepancies between local and testnet values ($<1\%$) arise from minor differences in block context (e.g., BASEFEE) that affect a small number of opcodes; they validate the equivalence of local development as a proxy for testnet behavior.

At the Base Sepolia gas price observed during our experiment (0.006 gwei), the full task lifecycle costs:

$$408,870 \times 0.006 \times 10^{-9} \text{ ETH} \approx 2.45 \times 10^{-6} \text{ ETH}$$

At an ETH price of \$3,000 USD, this is approximately **\$0.0074 per task delegation**—well within the range of commercial viability for agent services priced at cents to dollars.

The per-stage Gas breakdown also reveals a useful design validation: the REGISTER stage is the most expensive because it writes a variable-length JSON string on-chain, while SETTLE is cheapest because it mainly reads existing state and

transfers ETH. This aligns with our design goal of minimizing per-task overhead by amortizing registration costs across many task executions.

C. Experiment 3: Trust Assumption Analysis

Goal. Formally enumerate and compare the trust assumptions required by MAEP, x402, Google A2A, and Fetch.ai.

Method. For each protocol we enumerate trust requirements along four dimensions: agent identity, payment, result verification, and dispute resolution. A “trust assumption” is any entity whose honesty or availability must be assumed for the protocol’s security properties to hold.

Results. Table VI summarizes the comparison.

TABLE VI
TRUST ASSUMPTION COMPARISON ACROSS A2A PROTOCOLS

Dimension	x402	Google A2A	Fetch.ai	MAEP
Agent Identity	HTTP server	Google platform	Validator set	Ethereum contract (public)
Payment	Stablecoin issuer + server	None	FET + validators	Ethereum contract
Result Verification	None	None	Fetch.ai network	Ethereum contract
Dispute Resolution	None	Platform operator	Fetch.ai governance	AuditorAgent (configurable)
Trust Count	3	2	2	1

Analysis. MAEP achieves the lowest trust assumption count (1) among the surveyed protocols. The single remaining assumption—trust in Ethereum consensus—is shared with the entirety of the Ethereum DeFi ecosystem, which has processed trillions of dollars in value. It is a public, permissionless trust base with over 900,000 active validators as of 2025.

x402 requires three distinct trusted parties: the HTTP server (for honest accounting), the stablecoin issuer (for USDC redemption guarantees), and implicitly the payment processor routing the transaction. A compromise of any one breaks payment safety.

Google A2A’s platform trust assumption is qualitatively different from blockchain trust: it relies on Google’s organizational integrity and continued operation, which are subject to business and regulatory risk.

Fetch.ai’s validator set trust is formally similar to Ethereum’s, but the set is smaller (permissioned), making it more susceptible to coordinated attacks. Furthermore, Fetch.ai’s governance assumption adds a social-consensus dependency that MAEP avoids.

The AuditorAgent in MAEP’s dispute path is acknowledged as a trust assumption, but it is architecturally pluggable—it could be replaced by a multi-sig DAO, a cryptographic proof system, or eliminated for protocols where disputes trigger automatic refund.

D. Experiment 4: End-to-End Scenario Demonstration

Goal. Demonstrate that MAEP’s SDK enables a real three-agent system to execute both a cooperative and a contested task delegation end-to-end.

Setup. Three agents are instantiated: RequesterAgent (issues task), ProviderAgent (executes with LLM back-end), and AuditorAgent (resolves disputes). All agents use a locally running `gpt-oss:20b-cloud` model via the Ollama interface. Two scenarios are run:

- **Scenario A (Happy Path):** The requester delegates a data analysis task (“Analyze: 100 users, avg age 32, 60% male. Provide 3 key insights.”). The provider generates insights, posts the result hash on-chain, and the requester verifies and accepts.
- **Scenario B (Dispute Path):** The requester delegates analysis of an empty dataset. The provider returns a technically valid response (“dataset is empty—no analysis possible”). The requester rejects the result. The AuditorAgent reviews both the task specification and the result and issues a ruling.

Results.

TABLE VII
END-TO-END SCENARIO OUTCOMES

Scenario	Final Stage	Duration (s)	Ruling
A: Happy Path	SETTLED	1.84	–
B: Dispute	DISPUTED	4.20	PROVIDER

Analysis. Scenario A completes in 1.84 seconds end-to-end, the majority of which is LLM inference time. Protocol overhead (state transitions, hash computation) is unmeasurably small relative to inference latency, consistent with Experiment 1.

In Scenario B, the AuditorAgent rules in favor of the **Provider** (“PROVIDER” ruling), correctly identifying that the provider’s response is technically valid given the empty dataset, and that the requester’s rejection is unreasonable. This demonstrates that the dispute-resolution layer can exercise independent judgment rather than simply siding with the payer. The 4.20-second duration reflects two sequential LLM calls (provider execution + auditor evaluation).

Together, the two scenarios confirm that MAEP’s SDK correctly implements the state machine transitions `DELEGATED` → `EXECUTED` → `SETTLED` and `DELEGATED` → `EXECUTED` → `DISPUTED`, and that the AuditorAgent’s LLM-driven arbitration produces sensible results on a qualitative task.

VII. DISCUSSION

A. Limitations

Off-chain result delivery. MAEP commits a result *hash* on-chain but delivers the full result off-chain. If the provider sends a hash that does not match the off-chain data, the requester can detect the mismatch and dispute, but cannot recover the correct result on-chain. A future extension could use IPFS

content-addressed storage so that `ipfsCID(result)` is simultaneously the hash and the retrieval address.

Integrity vs. correctness. MAEP’s `ResultAttestation` contract verifies *integrity*: it guarantees that the result the requester received is exactly what the provider committed on-chain, with no post-hoc tampering. It does *not* verify *correctness*: a provider can submit a cryptographically valid hash whose pre-image is a low-quality, incomplete, or entirely wrong answer. The on-chain contract has no way to evaluate semantic content.

This gap is fundamental to any hash-commitment scheme and represents the primary trust residual in MAEP’s design. Three mitigation strategies exist, in increasing order of trustlessness:

- 1) **LLM Auditor (current).** The `AuditorAgent` reads both the task specification and the result and issues a ruling. This is practical for open-ended tasks (e.g., text analysis, summarization) where quality is inherently subjective, but the ruling is non-deterministic and depends on the auditor’s LLM backend.
- 2) **Verifiable task specification.** If the task can be expressed as a machine-checkable predicate—e.g., “sort this list”, “execute this SQL query”, “pass these unit tests”—the requester can verify correctness locally and deterministically without any auditor. The protocol would require the provider to attach an execution trace or test-pass certificate alongside the result hash. This approach works for structured, computational tasks but is inapplicable to generative or open-ended ones.
- 3) **Zero-knowledge proofs (future work).** A ZK-SNARK or ZK-STARK proof can attest that a computation was performed correctly without revealing the computation’s internals. For example, a provider could prove on-chain that a neural network inference on a given input produced a given output, enabling fully trustless result acceptance even for generative tasks. Current ZK proving costs for LLM-scale computations remain prohibitive, but are an active research area [9].

Auditor subjectivity. The `AuditorAgent`’s ruling is LLM-generated and therefore non-deterministic. Different runs may produce different rulings for the same dispute. For high-stakes use cases, the auditor should be replaced by a deterministic oracle or a multi-party consensus mechanism. The integrity-vs.-correctness limitation above subsumes this concern for the general case.

Testnet vs. mainnet. Our Gas cost analysis uses Base Sepolia. Mainnet Base has higher transaction throughput but also higher absolute gas prices at peak demand. The relative Gas efficiency of MAEP’s design holds regardless of absolute price, but practitioners should monitor L2 pricing trends.

Limited capability matching. The current DISCOVER stage performs exact string matching on `task_types`. Real deployment would benefit from semantic matching (embedding similarity) or structured capability schemas.

B. Future Work

- **Forced dispute on silence.** Currently a requester can silently withhold payment until the deadline without triggering arbitration. A future version could require the requester to explicitly accept or reject within a response window; silence after the window would auto-trigger the `AuditorAgent` or auto-release payment to the provider, closing the liveness gap identified in §IV.
- **ZK-proof result verification.** Replace the hash commitment with a zero-knowledge proof that the result satisfies a verifiable specification (e.g., a SQL query result, a sorting correctness predicate), enabling fully trustless result acceptance without any auditor.
- **Multi-agent task decomposition.** Extend the single Requester–Provider bilateral protocol to support a directed acyclic graph (DAG) of sub-tasks, where intermediate agents act as both providers (upstream) and requesters (downstream).
- **Reputation staking and slashing.** Allow third parties to stake on an agent’s reputation, creating financial incentives for honest behavior and a market for reputation insurance.
- **Cross-chain deployment.** Abstract the contract layer to support deployment on multiple EVM chains simultaneously, with cross-chain payment via bridge protocols.

VIII. CONCLUSION

We presented MAEP, a Minimum Viable Agent-to-Agent Economic Protocol that equips autonomous AI agents with on-chain decentralized identity, trustless escrow payment, and cryptographic result attestation. By implementing these primitives as three composable Ethereum smart contracts and an LLM-agnostic Python SDK, MAEP demonstrates that trust-minimized agent economies are achievable today without specialized infrastructure.

Our experimental evaluation shows that MAEP reduces the trust assumption count to 1 (Ethereum consensus) versus 2–3 for existing protocols; incurs a total on-chain Gas cost of approximately 408,870 units per task delegation—equivalent to less than one US cent at current Layer-2 prices; and adds only microsecond-level protocol overhead above plain function calls. An end-to-end three-agent demonstration, including a disputed task that is correctly adjudicated by an LLM-powered auditor, completes in under five seconds.

We hope MAEP serves as a useful reference point for the emerging field of agent economics, and invite the community to extend and critique the design.

ACKNOWLEDGMENTS

The authors thank the open-source communities behind Hardhat, web3.py, and Ollama for making rapid blockchain prototyping accessible.

REFERENCES

- [1] J. S. Park, J. C. O’Brien, C. J. Cai, M. R. Morris, P. Liang, and M. S. Bernstein, “Generative agents: Interactive simulacra of human behavior,” in *Proc. UIST*, 2023.

- [2] M. Sporny, D. Longley, M. Sabadello, D. Reed, O. Steele, and C. Allen, "Decentralized identifiers (DIDs) v1.0," W3C Recommendation, Jul. 2022. [Online]. Available: <https://www.w3.org/TR/did-core/>
- [3] Google LLC, "Agent2Agent (A2A) Protocol," 2025. [Online]. Available: <https://google.github.io/A2A/>
- [4] Coinbase, "x402: HTTP Payment Protocol," 2025. [Online]. Available: <https://www.x402.org/>
- [5] Fetch.ai Foundation, "Autonomous Economic Agents and the Fetch.ai Network," 2024. [Online]. Available: <https://fetch.ai/>
- [6] Valory AG, "Autonolas: On-chain Protocol for Co-owned AI," 2024. [Online]. Available: <https://olas.network/>
- [7] M. Andres et al., "Towards Multi-Agent Economies: Enhancing the A2A Protocol with Blockchain-Based Identities and Payments," arXiv preprint arXiv:2507.19550, 2025.
- [8] ConsenSys, "Ethereum Smart Contract Security Best Practices: Checks-Effects-Interactions," 2024. [Online]. Available: <https://consensys.github.io/smart-contract-best-practices/>
- [9] G. Wood, "Ethereum: A secure decentralised generalised transaction ledger," *Ethereum Project Yellow Paper*, vol. 151, 2014.
- [10] S. Nakamoto, "Bitcoin: A peer-to-peer electronic cash system," 2008. [Online]. Available: <https://bitcoin.org/bitcoin.pdf>
- [11] N. Szabo, "Formalizing and securing relationships on public networks," *First Monday*, vol. 2, no. 9, 1997.
- [12] V. Buterin, "Ethereum white paper: A next generation smart contract & decentralized application platform," 2013.